

Generic adaptive monitoring based on executed architecture runtime model queries and events

Thomas Brand and Holger Giese

Hasso Plattner Institute at the University of Potsdam, Germany

{firstname.lastname}@hpi.uni-potsdam.de

Abstract—Monitoring is a key functionality for automated decision making as it is performed by self-adaptive systems, too. Effective monitoring provides the relevant information on time. This can be achieved with exhaustive monitoring causing a high overhead consumption of economical and ecological resources. In contrast, our generic adaptive monitoring approach supports effectiveness with increased efficiency. Also, it adapts to changes regarding the information demand and the monitored system without additional configuration and software implementation effort. The approach observes the executions of runtime model queries and processes change events to determine the currently required monitoring configuration. In this paper we explicate different possibilities to use the approach and evaluate their characteristics regarding the phenomenon detection time and the monitoring effort. Our approach allows balancing between those two characteristics. This makes it an interesting option for the monitoring function of self-adaptive systems because for them usually very short-lived phenomena are not relevant.

I. INTRODUCTION

Advances in information technology allow capturing and processing vast and ever increasing amounts of data. This data is an important input for a variety of decisions, e.g., regarding system self-adaptation [1] or software evolution based on experimentation [2]. While technical constraints vanish, economical and ecological aspects become eventually significant. Both aspects require to focus on relevant information and therefore deciding which data to capture and especially which to process to meaningful information. But how to determine what is relevant and derive the monitoring configuration accordingly [3] when the monitored system and the information demands change over time [4]? Large scales, high speeds and frequencies of those changes suggest automating the finer-grained adaptation of monitoring mechanisms.

Generic adaptive monitoring solutions have the advantage of being reusable in different and changing contexts without significant development and configuration effort. In their systematic mapping Zavala et al. [5] point out that generic solutions are an interesting research opportunity and that only very few have been proposed so far. In our former work towards generic adaptive monitoring [6] we introduced the idea to observe query executions against the architecture runtime model of a system to determine the actual information demand and adapt the monitoring configuration accordingly. As before, we consider non-structural monitorable properties of the system, e.g., an operation invocation count, for the monitoring adaptation. However, our approach adapts inherently to structural system changes due to the causal connection between the runtime

model and the running system. In this paper we describe how effective monitoring, that detects relevant phenomena in time, can be ensured with our approach despite a reduced monitoring effort. For this purpose, we analyze different periodic and event-driven usage possibilities and report on experiments showing how each of those possibilities reduces the monitoring effort under different circumstances.

In section II we provide a brief introduction to monitoring and architecture runtime models before elaborating on the state of the art for adaptive monitoring. In section III we clarify the challenge that we want to tackle and with an example introduce further terms that help to explain how effective monitoring can be guaranteed. In section IV we describe our approach as well as different possibilities to use it and balance monitoring effort and phenomenon detection time. Afterwards we report on experiments and evaluate the results in section V. The conclusion and an outlook is provided in section VI. A detailed description of the design and approach implementation can be found in [7].

II. BACKGROUND

In this section we give a short introduction about monitoring, the state of the art, and architecture runtime models which are essential for our approach.

A. Architecture runtime models

"The basic underlying motivation for runtime models is the need for managing the complexity that arises from the large amounts of information that can be associated with runtime phenomena" [8]. A *runtime model* is an abstract causally connected representation of a running system. It allows monitoring and controlling the system as well as reasoning about it [9]. It can also serve as a knowledge base for self-adaptive systems. Formal modeling languages for runtime models, such as CompArch [10], enable the use of modeling techniques and tools from model-driven engineering to solve problems at runtime [11]. Runtime models that represent the actual architecture of a system are a typical use case [12]. Our approach relies on *architecture runtime models* to provide access to monitoring results.

B. Monitoring

Monitoring gathers and processes data to provide meaningful *monitoring results*. *Queries* can be used to access and further aggregate monitoring results. Phenomena that may also depend

on multiple conditions and the structure of a system can be detected with queries on the architecture runtime model of the system. The model represents all *monitorable properties* for accessing monitoring results. For the presented approach monitorable properties play an important role. They are similar to slots in the Unified Modeling Language (UML) and belong to a *monitorable object*, e.g., a particular component instance in a system. The *property specification* associated with the monitorable property determines the meaning of the monitoring results. The name monitorable property shall emphasize that in the case of adaptive monitoring they are not necessarily continuously monitored.

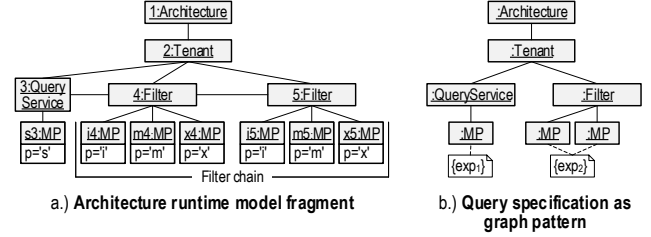
A *query specification* describes with a statement how a relevant *phenomenon* looks like or with an algorithm how it is detected. Such a phenomenon could, e.g., be that the performance of two related system components is reduced at the same time. A runtime model fragment that potentially constitutes a phenomenon instance but has not been fully evaluated is called *match candidate*. In comparison, a *match* is a detected phenomenon fulfilling the conditions defined in the query specification. Our approach observes which monitorable properties are accessed when a *query execution* processes a query specification.

C. State of the art

Moui et al. define adaptive monitoring in [13] as "the ability an online monitoring function has to decide and to enforce, without disruption, the adjustment of its behavior for maintaining its effectiveness, in respect of the variations of both functional requirements and operational constraints, and possibly for improving its efficiency according to self-optimization objectives." According to this definition, adaptive monitoring needs to adjust to changing requirements and constraints. Such requirements can be related to the current information demand or the monitored system. Also, constraints regarding response time determine whether adaptive monitoring is effective.

In their systematic mapping Zavala et al. [5] give an overview of the current state of the art on adaptive monitoring and categorize adaptive monitoring approaches into problem-specific, domain-specific and generic. Only 3 out of 81 approaches are generic. Those three focus on algorithms and rules as well as relationships among monitoring results to decide when to monitor what. None of them utilizes a runtime model to represent all possible monitorable properties and derives the monitoring configuration from observed queries, whose specifications describe the relevant information and allow balancing the monitoring effort and the phenomenon detection time as required.

Additionally, one can distinguish how the monitoring adaptation is performed. In the works of Villegas [14] and Tamura et al. [15] the monitoring is adapted in a coordinated manner by a higher-level adaptation engine together with the components that rely on the monitoring results. On the other hand, examples of monitoring self-adaption without such coordinating higher-level entity are the work of Munawar [16], who suggests to adapt monitoring for anomaly detection



Examples of a constraint expression (exp.) for a monitorable property (MP) with its property specification (p):

- 1.) self.p='s' and self.getMonitoringResult() = 'true'
- 2.) self.p='s' and self.getMonitoringResult('default=true') = 'true'
- 3.) self.p='s' and self.getMonitoringResult('default=false;changeNotifications=true') = 'true'

Fig. 1: Samples of a runtime model and a query specification

based on correlation information, and that of Ehlers [17], who employs a monitoring rule set to define which probes are active when. In contrast, our approach works without a coordinating higher-level adaptation engine, no limitation on a particular monitoring purpose, and no additional configuration effort.

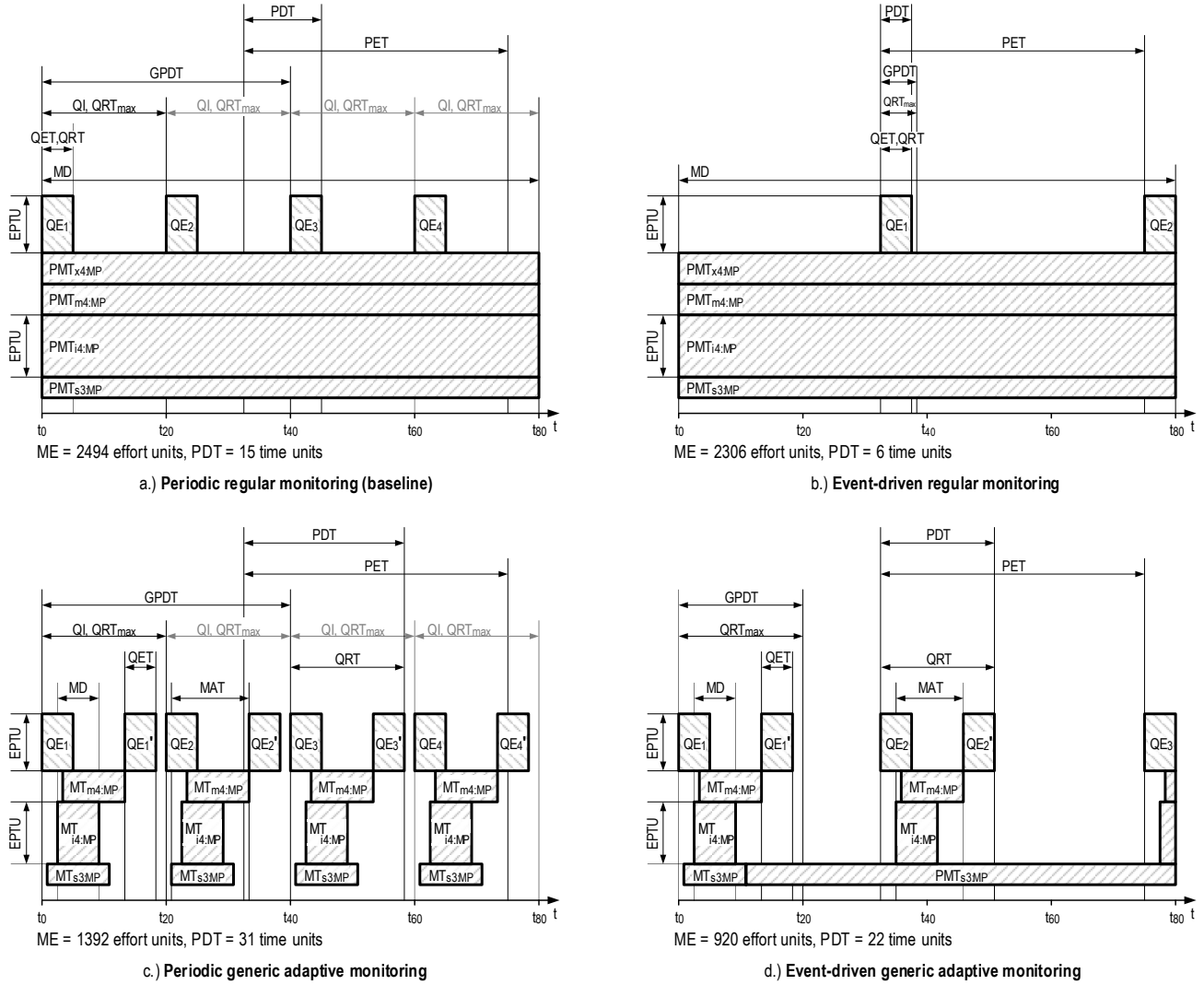
III. CHALLENGE AND EXAMPLE

Monitoring is performed to detect phenomena upon which a reaction is beneficial, e.g., to self-adapt a system to changing usage behavior. The challenge is to detect all relevant phenomena efficiently and within a required time frame because a phenomenon might be temporary or the reaction needs to happen within a certain period of time.

Our example is based on the SEAMS mRUBiS artifact [10], which mimics a multi-tenant online auctioning and sales system. A simplified fragment of the runtime model and an abbreviated query specification for illustration purposes is shown in Fig. 1. Each system tenant has a dedicated chain of ten filter components that personalizes for each customer the list of offered items, e.g., according to the customer's region and previous purchases. Depending on the set of available items, the order of the filters in the chain influences the overall response time. Thus, the order shall be checked when the performance decreases significantly. Each filter shall switch positions with its predecessor if the filter would save its predecessor on average more effort by discarding items than the predecessor does for that filter. To autonomously decide about such an adaptation, the query, e.g., first determines whether the monitoring result $r_{s3:MP}$ for the monitorable property of the *query service* (3) indicates a *significant performance decrease* (s). If this is the case, then the monitoring determines for two connected filters (4 and 5) the monitoring results $r_{i4:MP}$, $r_{m4:MP}$, $r_{i5:MP}$, and $r_{m5:MP}$ of the monitorable properties regarding the *items filtered on average* (i) and the required *milliseconds per item on average* (m). The position of the two filters shall then be switched if the condition for the following phenomenon is fulfilled: $r_{s3:MP} \ \& \ (r_{i5:MP} \times r_{m4:MP} > r_{i4:MP} \times r_{m5:MP})$.

IV. APPROACH AND POSSIBLE USES

With our approach, queries on the runtime model are used to detect phenomena. Those queries are triggered in repeating



Legend

GPDT Guaranteed phenomenon detection time
MAT Monitoring activation time
MD Monitoring duration
 $MT_{(p,q):MP}$ Monitoring task for monitorable property (MP) with monitorable object o and property specification p
PMT... Permanent monitoring task, index see MT

PDT Phenomenon detection time | $PDT \leq GPDT$
PET Phenomenon existence time | $PET_{min} \geq GPDT$
 $QE_{(q)}$ Query execution for query q
QET Query execution time
QI Query interval
QRT Query response time | $QRT_{max} \leq QI$

EPTU Effort per time unit
ME Monitoring effort | $ME = MTE + QEE$
MTE  Monitoring tasks effort
QEE  Query executions effort

Fig. 2: Possibilities to use the presented approach illustrated for one phenomenon occurrence

query intervals if the monitoring is *periodic*. In contrast, if the monitoring is *event-driven*, then they are triggered based on changes regarding monitoring results or the structure of the system. Preferably, incremental local queries are then triggered to keep the query effort low. If the monitoring is *adaptive* then the queries also activate the monitoring for monitorable properties on demand. Depending on the query specification, the activation can be for a single monitoring result or permanent. In contrast, *regular* monitoring is activated immediately and permanently without running queries.

With adaptive monitoring, the result of a query execution can be invalid if it is based on one or more preliminary monitoring results. A preliminary result, e.g., a default value as specified in the second constraint expression in Fig. 1, is provided by the monitoring during the query evaluation if the actual result still has to be produced. That way a missing monitoring result does not abort a query execution which therefore can continue to activate monitoring by accessing further monitorable properties. This usually reduces the number of required successive query executions to eventually produce

a valid query result and shortens the *monitoring activation time*. A query execution produced a valid query result if it only used actual monitoring results or if the provided preliminary monitoring results equal the actual results. A corresponding check is performed immediately after the missing monitoring results are available. The outcome of the check determines if another query execution is required for the query.

The *guaranteed phenomenon detection time* is the maximum allowable duration for detecting a new phenomenon occurrence because of the required reaction time or the minimum *phenomenon existence time*. Thus, for effective event-driven monitoring the maximum *query response time*, which comprises all durations including the query execution times to produce a valid query result, must not exceed the guaranteed phenomenon detection time as also shown, e.g., in Fig. 2d. In contrast, according to the sampling theorem two valid query results are required for effective periodic monitoring within the guaranteed phenomenon detection time. This is as also shown, e.g., in Fig. 2a.

We aim to keep the *phenomenon detection time* short and reduce the *monitoring effort*. The latter comprises the *query executions effort* as the summation of the efforts for all query executions in the period under consideration plus the *monitoring tasks effort* which is the accumulated efforts for the *monitoring tasks* that provide a single monitoring result and the *permanent monitoring tasks* that continuously produce results and notify about value changes. The size of the hatched areas in Fig. 2 indicate the extents of the two different kinds of effort. For illustration purposes we assume that for each query execution, monitoring task, and permanent monitoring task the effort increases proportionally to the *query execution time* respectively to the *monitoring duration* and that for all of them the specific *effort per time unit* is constant over time.

A. Periodic regular monitoring (baseline)

Shown in Fig. 2a, *periodic regular monitoring* is the baseline for our comparison of the four presented possibilities to use our approach. The queries to detect phenomena are triggered in repeating intervals. According to the sampling theorem, the guaranteed phenomenon detection time needs to be twice the query interval. The monitoring for all monitorable properties is immediately and permanently active and no query specific monitoring activation is required. Every query execution produces a valid query result and the query execution time is the query response time. Thus, the phenomenon detection time is shorter than with the periodic generic adaptive monitoring.

B. Event-driven regular monitoring

Shown in Fig. 2b, *event-driven regular monitoring* also activates the monitoring immediately and permanently. Thus, the time required for a single query execution is the query response time. As every change of a monitoring result or the system structure directly triggers a query, the guaranteed phenomenon detection time can be as short as the query response time assuming that the change notifications happen in no time.

C. Periodic generic adaptive monitoring

Shown in Fig. 2c, *periodic generic adaptive monitoring* has a guaranteed phenomenon detection time of two query intervals. The monitoring activation happens on demand and multiple query executions, e.g., QE_1 and QE_1' , might be required to produce a valid query result. Thus, the query response time and the phenomenon detection time are longer than with periodic regular monitoring while the monitoring tasks effort is lower.

D. Event-driven generic adaptive monitoring

Shown in Fig. 2d, *event-driven generic adaptive monitoring* first performs a query to activate permanent monitoring tasks, e.g., $PMT_{s3:MP}$. The user requests permanent monitoring and change notifications in the query specification as shown in the third constraint expression listed in Fig. 1. When the monitoring results of those tasks change, queries to detect phenomena are triggered which might activate further monitoring on demand. This leads to a longer query response time and phenomenon detection time than with event-driven regular monitoring. The monitoring tasks effort, however, is again lower.

E. Query evaluation

How a query is evaluated during its execution influences the monitoring activation and effort of our adaptive monitoring. When standard evaluation is performed, the monitorable properties are accessed and evaluated even if they cannot influence the query result anymore, e.g., if the condition for another monitorable property of the same match candidate was checked negatively already. The resulting monitoring activations cause unnecessary overhead but are beneficial for a short monitoring activation time and phenomenon detection time. This is the case because all monitorable properties that are required for a valid query result are activated during the same query execution even if the provided preliminary monitoring results do not fulfill the corresponding conditions.

When short-circuit evaluation is performed during a query execution, only monitorable properties are accessed and evaluated which can still influence the query result. This avoids unnecessary monitoring overhead and can lead to a shorter but also to a longer phenomenon detection time. It can be longer due to an iterative monitoring activation that is only based on actual monitoring results with a great query nesting depth. The more preliminary results, e.g., default values, however, fulfill the conditions for the corresponding monitorable properties, the shorter the monitoring activation time gets.

Additionally, it is also possible to combine periodic and event-driven generic adaptive monitoring. For monitorable properties with short response time requirements permanent monitoring tasks can be submitted to trigger queries immediately. Additional queries could be triggered periodically maybe then with a longer query interval.

V. EXPERIMENTS AND EVALUATION

To evaluate our approach, we conducted controlled experiments to compare the different possible uses described in

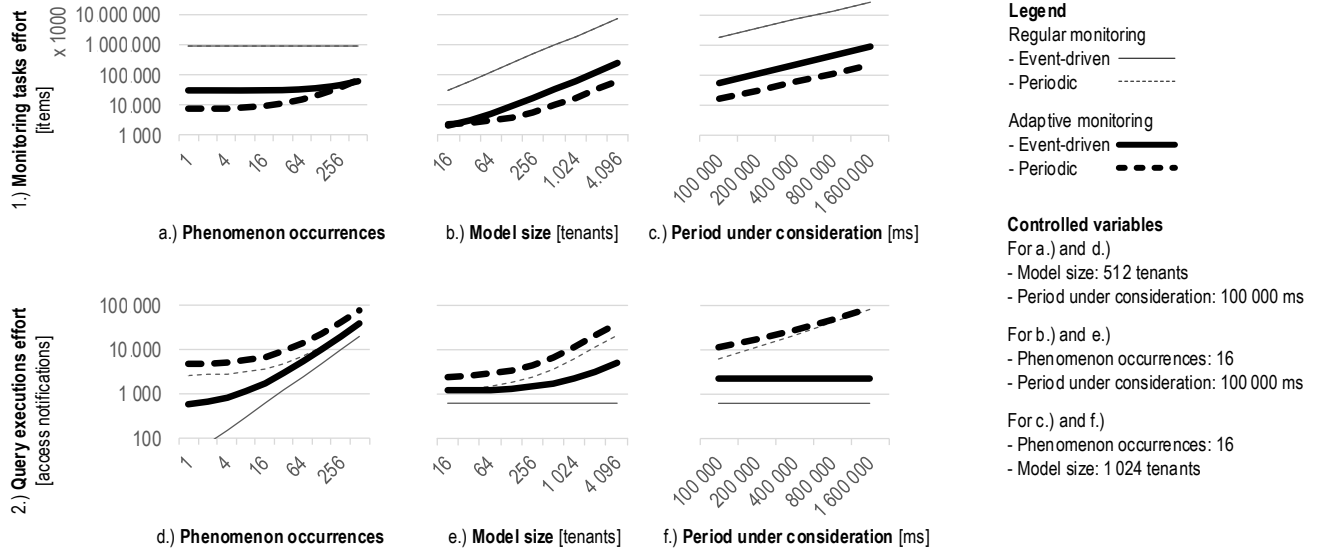


Fig. 3: Comparison of possibilities to use the presented approach based on experiment results

section IV. For this purpose we developed an implementation of our approach as a proof of concept with a simulated architecture runtime model and replayable monitoring results. Under the prerequisite that all phenomena are detected in time, the dependent variables that we are interested in are 1.) the monitoring tasks effort determined by the monitoring duration multiplied by the effort per time unit and 2.) the query executions effort indicated by the number of produced access notifications. Both can be easily and unbiasedly determined. As also depicted in Fig. 3, our independent variables are a./d.) the number of phenomena occurrences, b./e.) the runtime model size determined by the number of system tenants, and c./f.) the length of the period under consideration.

The time unit for the experiments is milliseconds. Further, all monitoring tasks have the same monitoring effort per time unit and the same monitoring duration. This makes it also easier to verify the measured results. The monitorable properties with the property specification x from Fig. 1 are also considered and add a constant amount of monitoring tasks effort for the regular monitoring. The results of our experiments are shown in Fig. 3 with logarithmic scales. First, we discuss the behavior of the monitoring task effort that is depicted in Fig. 3a-c before we turn to the query executions effort in Fig. 3d-f.

A. Monitoring task effort

As shown in Fig. 3a, the effort required by periodic and event-driven regular monitoring is independent from the number of phenomenon occurrences. In both cases all possible monitoring is active the whole time. The efforts for periodic and event-driven generic adaptive monitoring grow until the point that the monitoring is active all the time for the relevant monitorable properties. For low numbers of phenomenon occurrences, some rather constant monitoring effort is still significant for

the adaptive monitoring. This effort includes the one caused by permanent monitoring tasks for event-driven adaptive monitoring and the checking of potential but not complete match candidates which occurs frequently with periodic adaptive monitoring. In Fig. 3a the permanent effort for event-driven adaptive monitoring is greater than that for periodic adaptive monitoring. This can be caused by a permanent monitoring task like $PMT_{s3:MP}$ in Fig. 2d that requires more effort than the frequent executions of the monitoring task $MT_{s3:MP}$ triggered by periodic adaptive monitoring in Fig. 2c. Which adaptive monitoring requires less of the rather constant monitoring effort is situation specific and depends also on factors like the periodic query interval. Which adaptive monitoring possibility requires less effort can actually change for Fig. 3a-e.

As shown in Fig. 3b, the efforts for periodic and event-based regular monitoring grow linearly with the runtime model size. The efforts for periodic and event-based generic adaptive monitoring lie below those efforts and also grow linearly after the constant effort loses its significance for larger model sizes. In this case the constant effort stems from the monitoring that gets activated with the short-circuit evaluation to detect the occurred phenomena. As a controlled variable, the number of phenomena here stays the same and so this effort is constant.

As shown in Fig. 3c, for all monitoring possibilities the monitoring effort grows linearly with the period under consideration which is the time while monitoring tasks effort gets accumulated. It is remarkable that the effort for the adaptive monitoring is again clearly below that of both regular monitoring possibilities.

B. Query execution effort

As shown in Fig. 3d-f, the query executions effort for event-driven regular monitoring only changes, linearly, with the number of phenomenon occurrences. As shown in Fig. 3d, for

the other three monitoring possibilities the effort also grows linearly with larger numbers of phenomenon occurrences. For smaller numbers the growth is still influenced by the effort to check all potential match candidates respectively to activate permanent monitoring tasks.

As shown in Fig. 3e, the effort also grows linearly for the three monitoring possibilities with larger runtime model sizes. This is due to the activation of permanent monitoring tasks or the frequent checks for potential match candidates. For smaller model sizes the growth is still influenced by the effort to detect phenomenon matches. It is noteworthy that the effort for regular monitoring is lower because no access notifications are required to activate the monitoring and one query execution is always sufficient to produce a valid query result.

As shown in Fig. 3f, the period under consideration has no impact on the query executions effort for the event-driven regular and the generic adaptive monitoring. The reason for this is that in the experiment a longer duration does not mean more phenomena occurrences. In both cases queries are only triggered by monitoring result change notifications, e.g., when a phenomenon occurs or disappears. For event-based adaptive monitoring there is also the initial query to activate the permanent monitoring tasks for issuing the change notifications. The overall query executions effort growth of the periodic adaptive monitoring is notably influenced for shorter periods under consideration by the effort spent for monitoring activation and confirming phenomenon matches. But eventually the growth gets linear as for periodic regular monitoring. This experiment illustrates how event-driven monitoring can save significant amounts of query executions effort especially in the long run.

VI. CONCLUSION AND FUTURE WORK

The monitoring effort can significantly be reduced with our generic adaptive monitoring approach as conducted controlled experiments with a proof of concept implementation show. Our approach is independent of the monitoring purpose and supports different usage possibilities to balance between monitoring effort and phenomenon detection time as required to ensure monitoring effectiveness and achieve efficiency. This makes our approach an interesting option for providing the monitoring function of self-adaptive systems because very short-lived phenomena are usually also not relevant for system adaptations. Directions for future work are to enhance the monitoring adaptation regarding decision making, to elaborate on structural monitorable properties in the architecture runtime model and to explore additional use cases.

ACKNOWLEDGMENT

We thank Thomas Vogel for providing the reusable mRUBiS SEAMS exemplar and Matthias Barkowsky for his support with the Story Diagram Interpreter, used for the model queries.

REFERENCES

- [1] R. de Lemos, H. Giese, *et al.*, “Software engineering for self-adaptive systems: A second research roadmap,” in *Software Engineering for Self-Adaptive Systems II*, 2013.

- [2] J. Bosch, *Speed, Data, and Ecosystems - Excelling in a Software-Driven World*. CRC Press, 2017.
- [3] Y. Brun, R. Desmarais, *et al.*, “A design space for self-adaptive systems,” in *Software Engineering for Self-Adaptive Systems II*, 2013.
- [4] N. M. Villegas, G. Tamura, *et al.*, “Dynamico: A reference model for governing control objectives and context relevance in self-adaptive software systems,” in *Software Engineering for Self-Adaptive Systems II*, 2013.
- [5] E. Zavala, X. Franch, *et al.*, “Adaptive monitoring: A systematic mapping,” *Information and Software Technology*, 2018.
- [6] T. Brand and H. Giese, “Towards Generic Adaptive Monitoring,” in *Proceedings of the 12th International Conference on Self-Adaptive and Self-Organizing Systems (SASO)*, 2018.
- [7] T. Brand and H. Giese, “Generic adaptive monitoring with architecture runtime models,” Hasso Plattner Institute at the University of Potsdam, Germany, Tech. Rep., 2019, (to appear).
- [8] D. Weyns, “Software engineering of self-adaptive systems: An organised tour and future challenges,” in *Handbook of Software Engineering*, 2018.
- [9] G. Blair, N. Bencomo, *et al.*, “Models@run.time,” *Computer*, 2009.
- [10] T. Vogel, “mRUBiS: An exemplar for model-based architectural self-healing and self-optimization,” in *Proceedings of the 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, 2018.
- [11] H. Giese, T. Vogel, *et al.*, “State of the art in architectures for self-aware computing systems,” in *Self-Aware Computing Systems*, 2017.
- [12] N. Bencomo, S. Götz, *et al.*, “Models@run.time: A guided tour of the state of the art and research challenges,” *Software & Systems Modeling*, 2019.
- [13] A. Moui, T. Desprats, *et al.*, “Information models for managing monitoring adaptation enforcement,” in *Proceedings of the 4th International Conference on Adaptive and Self-adaptive Systems and Applications (ADAPTIVE)*, 2012.
- [14] N. Villegas, “Context management and self-adaptivity for situation-aware smart software systems,” PhD thesis, University of Victoria, 2013.
- [15] G. Tamura, N. M. Villegas, *et al.*, “Improving context-awareness in self-adaptation using the dynamico reference model,” in *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, 2013.
- [16] M. A. Munawar, “Adaptive Monitoring of Complex Software Systems using Management Metrics,” PhD thesis, University of Waterloo, 2009.
- [17] J. Ehlers, “Self-adaptive performance monitoring for component-based software systems,” PhD thesis, University of Kiel, 2012.